



Code Matters!

Frank Buschmann and Jörg Bartholdt

TIME AND AGAIN, the software community discusses whether architects should code.¹ Although broad consensus exists that they should, some voices argue that architects' responsibility for entire systems and their architectural integrity doesn't necessarily require coding.² At least these responsibilities limit the time architects can spend writing code. But how can architects balance coding activities with their other duties? And because all coding should have a clear contribution to their architectural work, how can they ensure that they code at appropriate "places" in the architecture, addressing architecturally significant aspects? Let's explore.

Where Architecture and Code Meet

A key aspect of architecture is to guide a system's implementation. All code artifacts should adhere to a common vision, interact effectively with one another, and comprise code that follows appropriate rules and guidelines. Architecture spawns an implementation space for a system, and it's the developer's task to choose a specific realization within that space. However, design and implementation aren't entirely independent from one another, and some realization options directly impact architecture.

The following code snippet outlines the signature of a service **TagManager** that manages the metadata of business logic objects in a map of so-called tags. Tags are defined as key value-pairs; they're used to advertise the metadata for other objects or the runtime to derive specific actions or adapt their behavior—for instance, to instantiate an object eagerly or lazily dependent on its current usage context:

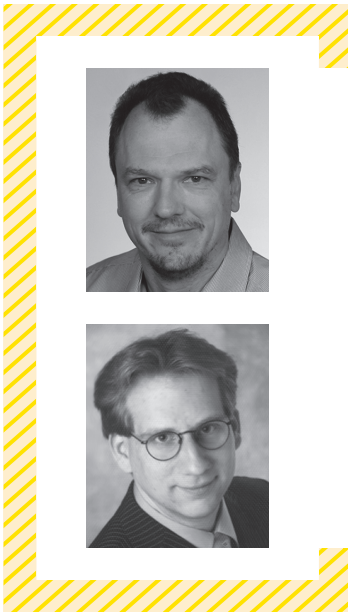
```
class TagManager {
    ...
    /** assign a string value to the identifier "tagName" */
    public void set(string tagName, string value);
    ...
}
```

After publishing the class interface, developers discovered that more operations were necessary to use it effectively—for instance, a method to remove a tag from the map. Unfortunately, modifying a released interface required going through a time-intensive change-request process. Developers thus tunneled the request for the remove operation through an existing method to the class implementation—which they could change without such ritual. In particular, they extended the parameter **tagName** of the **set** method with a prefix indicating that a remove operation should be performed:

```
tag.set("r:" + tagName, new string());
```

The implementation of the **set** method checked for the existence of the "r:" prefix in the **tagName** and removed the tag from the map if it was present, ignoring the second parameter of the call; otherwise, the method performed the original **set** operation. This programming trick didn't break existing code and let developers add new operations to the class without any interface changes. From a programmer's perspective, it was a reasonable response to avoid a complex and progress-blocking change request.

Actually, however, the developers implicitly modified the contract of the class and thus an aspect of the system's architecture—



although the class interface's signature didn't change. There were additional class responsibilities, explicit dependencies to a specific syntax for tag names, and increased complexity in the method's code and in the code calling the method. Developmental qualities thus suffered, most obviously maintainability, extensibility, and testability.

Code to Guide and Get Feedback

Ultimately, a system's architecture is defined by its code, not by its design models and diagrams. Bertrand Meyer, the creator of the Eiffel language, put it more vividly—"Bubbles don't crash"—therefore, "All you need is code!" Consequently, it's the architect's responsibility to pay attention to code—not just to avoid architectural drift and discover design flaws but also to identify coding aspects that impact architecture. The advice of the software community for addressing these issues is simple and direct: an architect also implements.³

While most architects generally accept this insight, the challenging question is this: How can they balance their responsibilities for the entire system with the need to stay in touch with development? If architects get lost in myriad local code details, it becomes difficult for them to define a sustainable architecture vision and drive its concrete design and realization. Yet, if they stay away from the system's code, they won't get feedback on their designs' habitability, nor will they discover the devil in the (code) details that's of architectural significance!

Consequently, all of an architect's coding activities must have a clear contribution to their main responsibility—the system's architecture, and its suitability to meet its requirements and business case, its architectural wholeness and integrity, and its usability for developers. Architects shouldn't code because they're the smartest developers on the team, want to demonstrate their coding skills, or just for fun—their ob-

jectives for programming should be to guide, educate, understand, and get feedback.

Programming to guide and educate teaches developers to focus on the architecture and design aspects whose realization is critical to success so that the architecture receives the architect's prominent attention. This practice tends to encourage developers to invest more care into writing idiomatic, high-quality code. Examples from our own coding practice include the intended use of important interfaces and the economic realization of critical usage scenarios.

Programming to understand and get feedback helps architects make pragmatic and sustainable design decisions and experience the consequences of their own designs. For example, a colleague recently focused his programming activities on exploring how to best avoid LOH (large object heap) fragmentation in .NET and thus design the system's data model and data management functionality accordingly. In general, this coding practice is extremely helpful to ground an architect's decisions and avoid the creation of design pearls that appear sound from a design perspective but are hardly usable in practice.

If the architects from our previous story had coded to "test" the usability of the `TagManager` class, they would have likely discovered its flaws and adjusted the interface before it was published for broad use. Thus, they would have avoided the developmental deficiencies caused by abusing an existing method signature for request tunneling.

If the architects had actively participated in coding, they would have likely discovered another issue with class `TagManager`. With the ability to set, change, remove, and observe tags, it became a fine-grained communication infrastructure between business logic objects. As tags got stored in a central process, even the communication of two business objects in the same process in-

involved two remote calls. Such usage led to significant performance degradation, jeopardizing the architecture's quality.

Agile Coding Practices

The focus on guidance and feedback also suggests programming practices that foster leadership and active programmer assistance. Architects should code to support developers in doing their job and enable them to make sustainable code-level decisions for their area of responsibility. Programming on the critical path or taking explicit ownership for the most challenging components is inappropriate for this purpose: Architects would dominate programming rather than guide it or even demonstrate mistrust in the developers' skills rather than empowering them to create economic code to benefit the system.

Two agile programming practices, pair programming and test-driven development (TDD), appear most suitable for leadership-oriented coding.

Pair programming lets architects program on all parts of the system—even on the critical path—without taking explicit ownership of the code.^{3,4} By pairing up with all developers on the team, they also get an overview of the entire code base and in-depth feedback on the habitability of their architecture. Through direct interaction with developers, they can give focused, concrete guidance on how to implement the architecture. Likewise, they can discover and address architecture drift, deficiencies, and code-level details of architectural relevance early and proactively. They can even improve their own coding skills when pairing with expert programmers.

Through programming tests, architects can guide the specification and realization of architectures from the "outside."⁵ Tests for end-to-end user scenarios, key quality attributes, and risky design and technology decisions stress architecture sustainability. Tests for component interfaces and externally

accessible APIs provide direct feedback on architecture usability. Pragmatic architects use the results of such tests to drive their own design work. The tests also form a framework for developers: they can program directly and explicitly toward defined architecturally relevant aspects.

Agile programming practices let architects control the time they spend on coding and move around to set a dedicated focus on the most important aspects of coding for the system's architecture. Development progress isn't dependent on the architect's constant presence in coding, and architects can balance their coding activities with all their other duties according to the architecture's current needs.

Code-level aspects like interface design and usage are often of architectural significance, requiring the architect's guidance and decisions. Consequently, coding isn't a complementary activity for architects but an integral part of a professional practice of their craft. Thus, architects should explicitly assign time for programming. However, they should be aware that programming isn't an end in its own right but one of many activities that guide and drive the design and realization of sustainable architectures. Then, coding isn't a price that architects must pay to win developers' respect but an important measure to ensure that they create sustainable designs that developers find habitable. ☞

architectsrule/archive/2010/05/07/do-architects-need-to-code.aspx.

3. J.O. Coplien and N. Harrison, *Organizational Patterns of Agile Software Development*, Prentice-Hall, 2004.
4. K. Beck, *eXtreme Programming Explained: Embrace Change*, Addison-Wesley, 2004.
5. F. Buschmann, "Tests: The Architect's Best Friend," *IEEE Software*, vol. 28, no. 3, 2011, pp. 7–9.

FRANK BUSCHMANN is a senior principal engineer at Siemens Corporate Technology, where he's a principal architect in the System Architecture and Platform Department. Contact him at frank.buschmann@siemens.com.

JÖRG BARTHOLDT is a senior engineer at Siemens Corporate Technology, where he's a senior certified software architect in the System Architecture and Platform Department. Contact him at joerg.bartholdt@siemens.com.

References

1. S. Brown, "Software Architecture for Developers," *Coding the Architecture*, online book; www.codingthearchitecture.com.
2. B. Vande Ghinste, "Do Architects Need To Code?" blog; [http://blogs.msdn.com/b/](http://blogs.msdn.com/b/architectsrule/archive/2010/05/07/do-architects-need-to-code.aspx)

Architecture doesn't stop when it comes to programming. Designs must be codified appropriately to avoid architecture drift.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

IEEE SOFTWARE CALL FOR PAPERS

Bridging Software Communities through Social Networking

SUBMISSION DEADLINE: 1 JUNE 2012 • PUBLICATION: JANUARY/FEBRUARY 2013

The advent of social networking sites has fundamentally altered how software development communities of stakeholders communicate, collaborate, learn from, and coordinate with one another. In many cases, social networking has eliminated boundaries and upended traditional hierarchies that constrain the flow of information within and between organizations. Networking technologies have enabled teams of developers to work together to produce software, groups of customers to easily share usage information, and developers and customers to exchange knowledge directly.

Our goal with this issue is to publish articles by researchers and practitioners who recognize and support connected communities of stakeholders at all scales. We hope that practitioners will gain knowledge on how social networking can play a role in all aspects of the software product life cycle—from conception and development to delivery, use, and reuse—and understand the potential advantages and possible challenges that lie ahead.

QUESTIONS?

For more information about the special issue, contact the guest editors:

- Jan Bosch, Professor, Chalmers University of Technology, Sweden, jan@janbosch.com
- Margaret-Anne Storey, Professor, University of Victoria, Canada, mstorey@uvic.ca
- Andrew Begel, Microsoft Research, USA, andrew.begel@microsoft.com

For the full call for papers: <http://www.computer.org/software/cfp1>

For author guidelines:

<http://www.computer.org/software/author.htm>

For submission details: software@computer.org

To submit an article:

<https://mc.manuscriptcentral.com/sw-cs>